Pipelines and Workflows
Software Engineering for Scientists

https://github.com/swe4s/lectures/tree/master/src/workflow/snakemake

The process of transforming data into a scientific result can involve many steps. These pipelines can be fragile and difficult to reproduce if executed manually. Workflow languages make this process more robust.

Install `snakemake`

```
$ conda activate swe4s
(swe4s) $ conda install -c conda-forge -c bioconda snakemake graphviz
```

Rules

A `snakemake` workflow is specified in a text file named `Snakefile` that contains a series of rules. Each rule is a small step in the process that takes a set of input files and runs a shell command to produce a set of output files.

From the `Snakefile`, `snakemake` will determine the dependencies between the rules by matching input files and output files and will execute the rules in an order that satisfies those dependencies. For example, if `rule A` has `foo.txt` as an input file, then `rule A` cannot execute until `foo.txt` exists. If `rule B` has `foo.txt` as an output file, then `rule B` may need to run before `rule A`. By tieing dependencies to files and not rules, `snakemake` can prevent wasted execution. If `foot.txt` already exists, then `rule B` does not need to run.

The input files, output files, and shell commands are specified in `input:`, `output:`, and `shell:` directives. The input and output directives contain Python lists of strings that resolve to paths to files. The shell directive contains a Python string that resolves to a bash command.

In the simple workflow below, a `plot.py` takes `foo.txt` as input and crates `foo.png`.

`Snakefile`

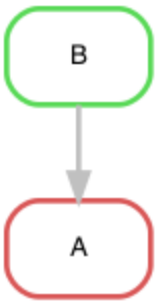
```
rule all:
  input:  "foo.png"

rule A:
  input:  "foo.txt"
  Output: "foo.png"
  shell:
    "python plot.py foo.txt foo.png"

rule B:
  output: "foo.txt"
  shell:
    "seq 1 100 > foo.txt"
```

The dependency graph can be visualized

```
$ snakemake --dag | dot -Tpng > dag.png
```



The steps that would be run in a workflow can also be inspected. When `--dryrun` is given none of the shell commands are actually exected.

```
$ snakemake --dryrun --printshellcmds
```

```
rule B:
    output: foo.txt
    jobid: 1

seq 1 100 > foo.txt

rule A:
    input: foo.txt
    output: foo.png
    jobid: 0

python plot.py foo.txt foo.png
Job counts:
      count jobs
      1     A
      1     B
      2
```

Once you are ready to execute the pipeline just run the `snakemake` command. In the output, yellow and green are good and red is bad.

```
$ snakemake

Provided cores: 1
Rules claiming more threads will be scaled down.
Job counts:
      count jobs
      1     A
      1     B
      2

rule B:
    output: foo.txt
    jobid: 1

Finished job 1.
1 of 2 steps (50%) done

rule A:
    input: foo.txt
    output: foo.png
    jobid: 0

Finished job 0.
2 of 2 steps (100%) done
```

If we look at the dependency graph again, we can see that it has been updated to show the rules whose dependencies have been met (output files exist) and do not need to be run.

```
$ snakemake --dag | dot -Tpng > dag.png
```



Specifying the runtime environment

`snakemake` rules also have a `conda:` directive that specifies an environment for the commands in the `shell:` directive. To use this feature, simply provide the path to a conda environment configuration file in the `conda:` directive. These files are in the `.yml` format and have a `channels:` section where you can give specific channels if you need packages that are not in the default channel, and a `dependencies:` section where you give the package names. While it is not required, specifying the version for these packages is highly recommended.

`Snakefile`

```
rule A:
  input:
    "foo.txt"
  output:
    "foo.png"
  conda:
    "env.yml"
  shell:
    "python plot.py foo.txt foo.png"
```

`env.yml`

```
channels:
  - conda-forge
  - bioconda
dependencies:
  - python=3.6.9
  - matplotlib=3.1.1
```

With the conda: directive and the conda environment file, pass the `--use-conda` flag to `snakemake`

```
$ snakemake --use-conda
```

Example workflow

Suppose we want to look at a histogram of the fires in different countries. First, we need to download the file containing the data.

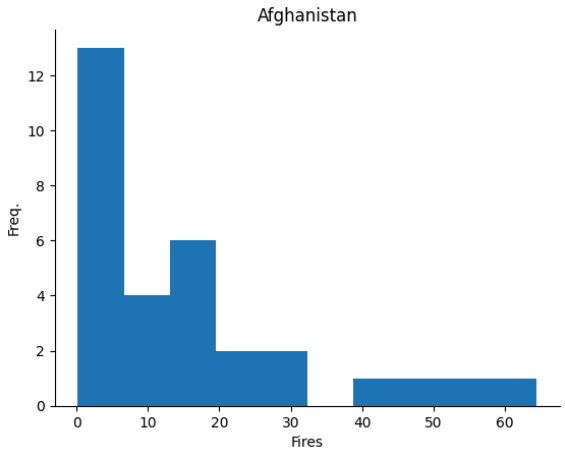
```
$ wget -O Argrofood_co2_emission.csv
'https://docs.google.com/uc?export=download&id=1Wytf3ryf9EtOwaloms8HEzLG0yjtRqxr'
```

Next, we need to extract the fire data for a given country and store it in a file.

```
$ python get_data.py Argrofood_co2_emission.csv Afghanistan Afghanistan.txt
```

Last we want to make a histogram of those counts. `hist.py` takes a file with one number per line and creates a histogram of that data.

```
$ python hist.py Afghanistan.txt Afghanistan.png Afghanistan Fires Freq.
```



`snakemake` specification

A simple workflow for getting the above plot would be

```
rule all:
  input:
    'Afghanistan.txt'

rule country_fire_counts:
  input:
    'Argrofood_co2_emission.csv'
  output:
    'Afghanistan.txt'
  shell:
    'python get_data.py {input} Afghanistan {output}'

rule country_plot_hist:
  input:
    'Afghanistan.txt'
  output:
    'Afghanistan.png'
  shell:
    'python hist.py {input} {output} Afghanistan Fires Freq.'
```

Here we take advantage of variables `{input}` and `{output}` that resolve to the values of the input and output directives. NOTE: `snakemake` abuses the strings wrapped in curly brackets syntax. In some instances they are variables and in others they are wildcards which we will cover next.

The issue with these rules can only create the country fire counts files for Afghanistan. With rules like these, we would need to create new rules for every country. To help make our rules more general, Snakemake uses *wildcards*. A wildcard is a placeholder used within rule definitions to generalize filenames and allow for pattern matching. Wildcards make rules more flexible and adaptable to different inputs and outputs without having to specify each filename explicitly. Wildcards are denoted by curly braces `{}` in rule definitions. For example, if we replace the output with a wildcard, then `country_file_counts` will work for any country.

```
rule country_fire_counts:
  input:
    'Argrofood_co2_emission.csv'
  output:
    '{x}.txt'
  shell:
    'python get_data.py {input} {wildcards.x} {output}'
```

When snakemake runs, we can see how the wildcard is set

```
rule country_fire_counts:
    input: Argrofood_co2_emission.csv
    output: Afghanistan.txt
    jobid: 2
    reason: Missing output files: Afghanistan.txt
    wildcards: x=Afghanistan
    resources: tmpdir=/var/folders/25/qkklxfbx5gg8dv9cppcq9c8h0000gn/T
```

Actually, `country_file_counts` will trigger for any string ending in `.txt`, which can lead to some ambiguity. For example, if we had some other rule that, for example, generated random input data.

```
rule random_file:
  output:
    '{x}.txt'
  shell:
    'for i in `seq 1 100`; do echo $RANDOM; done > {output}'
```

Snakemake would give the following error:

```
AmbiguousRuleException:
Rules random_file and country_fire_counts are ambiguous for the file Afghanistan.txt.
Consider starting rule output with a unique prefix, constrain your wildcards, or use
the ruleorder directive.
Wildcards:
      random_file: x=Afghanistan
      country_fire_counts: x=Afghanistan
Expected input files:
      random_file:
      country_fire_counts: Argrofood_co2_emission.csv
Expected output files:
      random_file: Afghanistan.txt
      country_fire_counts: Afghanistan.txt
```

To prevent this, you must differentiate the file name patterns. For example, make random files end in the pattern `.rand.txt` and make that data files end in `.data.txt`

```
rule country_fire_counts:
  input:
    'Argrofood_co2_emission.csv'
  output:
    '{x}.data.txt'
  shell:
    'python get_data.py {input} {wildcards.x} {output}'

rule random_file:
  output:
    '{x}.rand.txt'
  shell:
    'for i in `seq 1 100`; do echo $RANDOM; done > {output}'
```

Using the same logic, we can make `country_plot_hist` a more general rule

```
rule country_plot_hist:
  input:
    '{y}.txt'
  output:
    '{y}.png'
  shell:
    'python hist.py {input} {output} {wildcards.y} Fires Freq.'
```

With rules that can work for any country, we would then like to make it easier to extend our workflow to include other countries. The expand function can take a list of countries and create a list of files.

```
COUNTRIES = [
rule country_plot_hist:
  input:
    '{y}.txt'
  output:
    '{y}.png'
  shell:
    'python hist.py {input} {output} {wildcards.y} Fires Freq.'
```